ACTIVIDAD 2: EL TALLER DEL PROGRAMADOR

Laboratorio de Programación (LPR) — 5° 3° A-B

Autores:

Sofia Mojica

Gabriel Zacarias

Mateo Roberts

Leonel De Armas

Docentes a cargo: York Mansilla / Yamil Ganduglia

1. **Entornos de Desarrollo Seleccionados**

- **Visual Studio Code (VS Code):** Editor ligero y altamente personalizable mediante extensiones, consolidado como el estándar de la industria.
- **Visual Studio Community:** Un IDE robusto y completo, diseñado para proyectos de gran escala empresarial y desarrollo de escritorio complejo.
- **Sublime Text:** Un editor de texto sofisticado, caracterizado por su alta velocidad y eficiencia en la edición rápida de archivos independientes.

2. **Conceptos Clave del Laboratorio**

- **Código Fuente:** El conjunto de líneas de texto escritas en un lenguaje comprensible para el ser humano (C + +), el cual requiere traducción.
- **Compilador:** Herramienta de alto nivel que traduce el código fuente a código máquina (binario), generando un archivo ejecutable (`.exe`).
- **Depurador (Debugger):** Herramienta encargada de ejecutar el programa de forma secuencial y controlada para detectar fallas lógicas ("bugs").
- **Terminal/Consola:** Interfaz de texto plano para enviar comandos directos al sistema operativo.
- **Variables de Entorno (PATH):** Registro del sistema operativo donde se listan las rutas de los ejecutables del sistema (esencial para mapear `g + +`).

3. **Distribución de Roles y Tareas del Equipo**

Integrante	Rol / Tarea Asignada
Gabriel Zacarías	Configuración del entorno base, instalación de **MSYS2** y despliegue del compilador **MinGW (GCC/G++)**.
Sofía Mojica	Modificación del código fuente en **`src/main.cpp`** con los datos del equipo y pruebas de compilación en la terminal.

Mateo Roberts	Gestión de las evidencias visuales, toma de **capturas de pantalla** reglamentarias y actualización del `README.md`.
Leonel De Armas	Redacción del glosario técnico, diseño del **informe bajo normativa APA v7** y documentación en la Carpeta de Campo.

4. **Estructura de carpetas del proyecto**

/**HOLAMUNDO**/
├── .gitignore <-- Archivo para que Git ignore carpetas de configuración (.vscode)
├── README.md <-- Archivo explicativo del entorno que configuraron
├── LICENSE <-- Descripción breve de la licencia
├── docs/ <-- Carpeta exclusiva para documentación
│ └── InformeProyectoHOLAMUNDO.pdf <-- ENTREGABLE (En PDF con formato APA v7)
├── src/ <-- Carpeta para los códigos fuente
│ └── main.cpp <-- Primer código de prueba en C++
└── capturas/ <-- Carpeta para guardar las evidencias (Imágenes .png)
├── gpp_version.png <-- Captura de la consola verificando el compilador
├── extensiones_vscode.png <-- Captura de las extensiones instaladas
└── ejecucion_hola_mundo.png <-- Captura de la terminal compilando y corriendo el programa

5. **Cuestionario Técnico de Control (Guía de Respuestas)**

1. ¿Qué entorno de desarrollo te parece más intuitivo a primera vista? ¿Por qué?

El entorno más intuitivo es *Visual Studio Code*. Porque tiene una interfaz más limpia, su barra lateral está muy bien organizada para extensiones y control de versiones (Git), y su capacidad de integrar la terminal en la misma pantalla sin la necesidad de abrir otra ventana o de saturarnos a la hora de programar.

2. ¿Qué características de un IDE creés que te facilitan más el aprendizaje?

El **autocompletado de código**, por el resaltado de sintaxis por colores que nos ayuda a identificar errores visualmente, y la integración de una **terminal interna** que evita tener que alternar constantemente entre diferentes ventanas del sistema operativo.

3. De las tareas realizadas, ¿cuál te resultó más difícil de configurar?

La configuración y el mapeo de las **Variables de Entorno (PATH)** para el compilador MinGW/MSYS2, porque nos resultó muy tedioso y si tienes cualquier error en la sintaxis de la ruta o la falta de reinicio del IDE provoca fallas de reconocimiento del comando `g++` en la terminal.

4. ¿Qué diferencia técnica encontrás entre un "Editor de Texto" (como Sublime) y un "IDE" (como VS Community)?

Un editor de texto es puramente un lienzo ligero para modificar código plano de manera veloz. En contraste, un **IDE** viene con todo el ecosistema integrado de forma nativa: incluye su propio compilador, herramientas avanzadas de depuración (debugging), diseñadores gráficos de interfaces y gestores de servidores complejos.

5. ¿Qué tipo de aplicaciones sueñan con desarrollar al recibirse cuando lleguen a 6° y 7° año? ¿Por qué?

A nosotros nos gustaría desarrollar un videojuego, porque C++ es el lenguaje estándar de motores potentes como *Unreal Engine* y permite optimizar al máximo los recursos de hardware y también nos encantaría desarrollar aplicaciones web y móviles escalables utilizando arquitecturas modernas (como Flask/Python para Backend) para resolver problemas logísticos cotidianos de los usuarios.

6. ¿Qué otros lenguajes de programación te gustaría investigar además de C, C++? ¿Por qué?

Nos gustaría investigar y saber sobre **C# (C Sharp)**, debido a su fuerte integración con el entorno Microsoft para aplicaciones de escritorio y el desarrollo de videojuegos en Unity; o **Java**, por su amplio uso corporativo en sistemas bancarios y desarrollo nativo de aplicaciones Android.

7. ¿Qué tan importante considerás que es el orden y la nomenclatura (los nombres) de tus archivos y carpetas? ¿Por qué?

Es muy importante porque en el desarrollo profesional y el trabajo en equipo (Git), mantener estructuras estandarizadas (como separar `src/` de `docs/`) y nombrar archivos de forma descriptiva previene la pérdida de información, facilita el mantenimiento del código y asegura un flujo de trabajo limpio.

8. Si pudieras diseñar el "entorno ideal", ¿qué herramienta o funcionalidad le agregarías para que programar sea más fácil?

Sería muy bueno crear una herramienta nativa de **colaboración en tiempo real en la nube** como en docs de google que incluya un chat de depuración por voz integrado y un sistema automático de resolución de conflictos de ramas de Git y también que se pueda editar el código en tiempo real entre todos los integrantes.

9. ¿Cómo creés que la IA puede ayudar a un programador sin que este pierda su capacidad de razonamiento?

La Inteligencia Artificial se debe usar como un **asistente de soporte o copiloto**, delegando tareas repetitivas (como escribir documentación, estructurar un README o buscar errores de sintaxis). El programador no debe pedirle que resuelva la lógica del problema, sino usarla para entender alternativas de optimización, manteniendo siempre el control del diseño del algoritmo.

10. ¿Cuál es tu meta principal para este año en la materia LPR? ¿Por qué?

Nos gustaría poder dominar la transición entre la lógica de Python y la estructuración de bajo nivel de C++, asimilando las buenas prácticas de Git y el trabajo colaborativo en equipo, preparándonos para los desafíos técnicos de los proyectos complejos de 6° y 7° año y también para nuestro futuro laboral.

6. Conclusión

La realización de esta actividad permitió consolidar las bases técnicas del taller del programador. Comprender el funcionamiento interno detrás de la edición, compilación y enlace de rutas de sistema mitiga la dependencia de herramientas automatizadas y brinda autonomía técnica al estudiante. El despliegue exitoso del primer programa "Hola Mundo" en C + + valida que el entorno se encuentra en condiciones óptimas para afrontar los desafíos algorítmicos complejos.

7. **Enlaces y Aportes Individuales de Integrantes**

Cada integrante del grupo debe ingresar a su apartado correspondiente a continuación y colocar su link individual (repositorio de GitHub).

1. ***Leonel De Armas:***

https://github.com/tomasdearmas99-png/ACTIVIDAD-2-EL-TALLER-DEL-PROGRAMADOR---LPR-5-3-A-B.git

2. ***Sofia Mojica***

https://github.com/Ailuna77/Actividad-2-Hola-Mundo.git